

PoC – Experimento que separa processamento documental, geração de embeddings, armazenamento no Qdrant, retrieval por modelo e fusão de rankings.

1. Objetivo da POC

A POC tenta responder quatro perguntas:

1. Uma única collection pode ter embeddings de dimensões diferentes?
2. O mesmo documento pode armazenar vetores produzidos por modelos diferentes?
3. Como selecionar qual modelo usar durante o retrieval?
4. É possível trocar ou comparar modelos sem repetir parsing, OCR e chunking?

A resposta demonstrada é:

Uma collection

└─ named vector e5_small_384

└─ named vector e5_base_768

└─ named vector e5_large_1024

Cada named vector é um espaço vetorial independente, com dimensão e configuração próprias. Dentro de cada espaço, a dimensão permanece fixa.

[\[qdrant.tech\]](https://qdrant.tech), [\[qdrant.tech\]](https://qdrant.tech)

2. Visão geral do fluxo

O fluxo pode ser apresentado assim:

Documentos fictícios

|

v

Texto já processado

|

+-----+-----+

|||

v v v

E5 Small E5 Base E5 Large

384 dimensões 768 dimensões 1024 dimensões

|||

+-----+-----+

|

v

Um ponto na mesma collection

|

v

Payload compartilhado

Durante o retrieval:

Pergunta

|

+--> embedding E5 Small --> busca usando e5_small_384

|

+--> embedding E5 Base --> busca usando e5_base_768

|

+--> embedding E5 Large --> busca usando e5_large_1024

|

v

Resultados independentes

|

v

Reciprocal Rank Fusion

|

v

Ranking consolidado

A Query API permite indicar o named vector por meio de using, portanto cada consulta é direcionada ao espaço correspondente.

3. config.py: catálogo dos modelos

O arquivo contém uma estrutura semelhante a:

```
@dataclass(frozen=True)

class ModelSpec:

    vector_name: str

    model_id: str

    expected_dimension: int
```

Essa classe descreve um modelo de embedding.

Significado dos campos

1. vector_name

É o nome usado dentro da collection do Qdrant:

e5_small_384

e5_base_768

e5_large_1024

2. model_id

É o identificador do modelo no Hugging Face:

intfloat/multilingual-e5-small

intfloat/multilingual-e5-base

intfloat/multilingual-e5-large

3. expected_dimension

É a dimensão que esperamos receber do modelo:

384

768

1024

A configuração completa:

```
MODELS = (  
    ModelSpec(  
        "e5_small_384",  
        "intfloat/multilingual-e5-small",  
        384,  
    ),  
    ModelSpec(  
        "e5_base_768",  
        "intfloat/multilingual-e5-base",  
        768,  
    ),  
    ModelSpec(  
        "e5_large_1024",  
        "intfloat/multilingual-e5-large",  
        1024,  
    ),  
)
```

Por que centralizar isso?

Porque evitamos espalhar pelo código:

nome do modelo

nome do vetor

dimensão esperada

Se um modelo mudar, a alteração fica concentrada nessa configuração.

4. documents.py: dados fictícios e golden dataset

O arquivo possui duas estruturas.

Documentos fictícios

Exemplo:

```
1  {
2      "id": 2,
3      "title": "Prazo de implantação",
4      "department": "Projetos",
5      "text": (
6          "O prazo estimado para implantação e go-live "
7          "da solução é de até 30 dias."
8      ),
9  }
```

Esses documentos representam os conteúdos que seriam extraídos de PDFs, DOCX ou outras fontes no sistema real.

Na POC, o conteúdo já está limpo para isolar o experimento:

- não estamos testando OCR
- não estamos testando parsing
- não estamos testando chunking
- estamos testando embeddings e retrieval

Golden queries

Exemplo:

```
1  {
2      "query": "Qual o prazo para o go-live?",
3      "expected_id": 2,
4  }
```

A pergunta tem um documento esperado. Isso permite medir objetivamente o retrieval:

O modelo trouxe o documento correto?

Em qual posição?

Ele apareceu no top 3?

5. embeddings.py: carregamento e geração dos vetores

A classe principal é:

```
class EmbeddingRegistry:
```

Ela administra os três modelos.

Cache dos modelos

```
self._models: dict[str, SentenceTransformer] = {}
```

Isso evita carregar o mesmo modelo toda vez que uma pergunta é processada.

O método:

```
def _model(self, spec: ModelSpec) -> SentenceTransformer:
```

verifica:

```
if spec.vector_name not in self._models:
```

Se o modelo ainda não estiver em memória, ele é carregado:

```
model = SentenceTransformer(  
    spec.model_id,  
    device="cpu",  
)
```

Depois, a dimensão real é validada:

```
dimension = model.get_sentence_embedding_dimension()  
  
if dimension != spec.expected_dimension:  
    raise ValueError(...)
```

Essa validação evita um problema perigoso:

configuração diz 768

modelo retorna 1024

upsert falha por dimensão incompatível

Embeddings dos documentos

```
1 def embed_passages(  
2     self,  
3     spec: ModelSpec,  
4     texts: list[str],  
5 ) -> list[list[float]]:
```

O prefixo utilizado é:

 passage: conteúdo do documento

 prepared = [
 f"passage: {text}"
 for text in texts
]

Depois:

```
    vectors = model.encode(  
        prepared,  
        normalize_embeddings=True,  
    )
```

A normalização é coerente com busca por cosseno.

Embedding da pergunta

```
1 def embed_query(  
2     self,  
3     spec: ModelSpec,  
4     query: str,  
5 ) -> list[float]:
```

Isso é importante porque os modelos E5 distinguem consultas de passagens.

6. qdrant_store.py: criação da collection

A classe:

Class MultiEmbeddingStore:

encapsula as operações do Qdrant.

Criação da collection

O ponto central da hipótese está aqui:

```
1  vectors_config={
2      spec.vector_name: models.VectorParams(
3          size=spec.expected_dimension,
4          distance=models.Distance.COSINE,
5      )
6      for spec in specs
7  }
```

Isso produz uma configuração equivalente a:

```
1  {
2      "e5_small_384": VectorParams(
3          size=384,
4          distance="Cosine",
5      ),
6      "e5_base_768": VectorParams(
7          size=768,
8          distance="Cosine",
9      ),
10     "e5_large_1024": VectorParams(
11         size=1024,
12         distance="Cosine",
13     ),
14 }
```

O Qdrant exige dimensão constante dentro de um mesmo vetor nomeado, mas named vectors diferentes podem ter dimensões diferentes.

7. Como cada ponto armazena três embeddings

No método:

upsert_documents(...)

é criado um dicionário de vetores:

```
1 point_vectors = {
2     vector_name: vectors[index]
3     for vector_name, vectors in vectors_by_name.items()
4 }
```

Para o documento id=2, o resultado conceitual é:

```
1 {
2     "id": 2,
3     "vector": {
4         "e5_small_384": [/* 384 valores */],
5         "e5_base_768": [/* 768 valores */],
6         "e5_large_1024": [/* 1024 valores */],
7     },
8     "payload": {
9         "id": 2,
10        "title": "Prazo de implantação",
11        "department": "Projetos",
12        "text": "O prazo estimado...",
13    },
14 }
```

A informação documental é armazenada uma vez no payload, enquanto as três representações vetoriais ficam associadas ao mesmo ponto.

8. Retrieval usando using

O método:

```
1 def search(
2     self,
3     vector_name: str,
4     query_vector: list[float],
5     limit: int = 5,
6 ):

```

executa:

```
1 result = self.client.query_points(  
2     collection_name=self.collection_name,  
3     query=query_vector,  
4     using=vector_name,  
5     limit=limit,  
6     with_payload=True,  
7 )
```

O campo-chave é:

using=vector_name

Exemplo para o modelo Small:

```
1 store.search(  
2     vector_name="e5_small_384",  
3     query_vector=query_vector_384,  
4 )
```

O Qdrant compara esse vetor apenas com:

e5_small_384

Para o Base:

```
1 store.search(  
2     vector_name="e5_base_768",  
3     query_vector=query_vector_768,  
4 )
```

O Qdrant compara apenas com:

e5_base_768

Não existe comparação direta entre um vetor de 384 e um de 768 dimensões. Cada named vector representa um índice ou espaço lógico independente, selecionado na consulta.

9. Por que não comparar os scores diretamente?

Cada modelo produz uma distribuição própria:

Small:

0.91

0.86

0.82

Base:

0.78

0.76

0.71

Large:

0.96

0.88

0.84

Não podemos concluir automaticamente que:

0.96 do Large é melhor que 0.91 do Small

Os valores pertencem a espaços vetoriais diferentes.

Por isso a POC usa **Reciprocal Rank Fusion**.

10. fusion.py: Reciprocal Rank Fusion

A fórmula do RRF é:

```
1 score(documento) += 1 / (k + posição)
```

No código:

```
scores[point_id] += 1.0 / (k + rank)
```

Se um documento ficar:

1º no Small

2º no Base

1º no Large

ele receberá:

1 / 61

+

1 / 62

+

1 / 61

Outro documento que aparecer apenas em um ranking terá pontuação menor.

Benefício

O RRF privilegia consenso entre modelos:

Documento bom em vários modelos

→ sobe no ranking consolidado

E não depende da escala bruta dos scores de similaridade.

11. evaluation.py: métricas

A POC calcula três métricas.

Accuracy@1

Mede quantas vezes o documento correto ficou na primeira posição.

Exemplo:

5 perguntas

4 documentos corretos em primeiro

$\text{Accuracy@1} = 4 / 5 = 0.80$

Recall@3

Mede quantas vezes o documento esperado apareceu entre os três primeiros.

5 perguntas

5 documentos dentro do top 3

Recall@3 = 1.00

MRR

Mean Reciprocal Rank:

1º lugar → $1/1 = 1.00$

2º lugar → $1/2 = 0.50$

3º lugar → $1/3 = 0.33$

Depois é calculada a média.

O MRR ajuda a distinguir:

modelo que encontra o documento sempre em primeiro

de:

modelo que encontra, mas frequentemente em segundo ou terceiro

12. run_poc.py: orquestração completa

O script principal conecta todas as peças.

Passo 1: clientes e serviços

```
client = QdrantClient(url=QDRANT_URL)
registry = EmbeddingRegistry(MODELS)
store = MultiEmbeddingStore(
    client,
    COLLECTION_NAME,
)
```

Responsabilidades:

client → comunicação com o Qdrant

registry → geração dos embeddings

store → persistência e busca

Passo 2: recriação da collection

```
store.recreate_collection(MODELS)
```

A POC remove a collection anterior para garantir um experimento reproduzível.

Em produção, não faríamos isso automaticamente.

Passo 3: geração dos embeddings

```
1 texts = [  
2     document["text"]  
3     for document in DOCUMENTS  
4 ]
```

Depois:

```
1 vectors_by_name = {  
2     spec.vector_name: registry.embed_passages(  
3         spec,  
4         texts,  
5     )  
6     for spec in MODELS  
7 }
```

Conceitualmente:

```
1 {  
2     "e5_small_384": [  
3         vetor_documento_1,  
4         vetor_documento_2,  
5         ...  
6     ],  
7     "e5_base_768": [  
8         vetor_documento_1,  
9         vetor_documento_2,  
10        ...  
11    ],  
12    "e5_large_1024": [  
13        vetor_documento_1,  
14        vetor_documento_2,  
15        ...  
16    ]  
17 }
```

Passo 4: upsert

```
1 store.upsert_documents(  
2     DOCUMENTS,  
3     vectors_by_name,  
4 )
```

São inseridos:

- 6 pontos

- 3 vetores por ponto

- 18 representações vetoriais

Passo 5: perguntas

Para cada golden query, o sistema executa os três modelos;

Para cada modelo:

1. Gera embedding da pergunta.
2. Consulta seu named vector.
3. Coleta os IDs retornados.
4. Localiza a posição do documento esperado.

Passo 6: fusão

```
fused = reciprocal_rank_fusion(rankings)
```

Os três rankings são combinados.

Passo 7: métricas

```
1 summary = {  
2     model: evaluate_rankings(items)  
3     for model, items in model_evaluations.items()  
4 }
```

Ao final, podemos comparar:

- E5 Small

E5 Base

E5 Large

Fusão RRF

13. Como interpretar seu retorno do Qdrant

Recebeu:

```
1  "status": "green",
2  "optimizer_status": "ok",
3  "indexed_vectors_count": 0,
4  "points_count": 6
```

status: green

A collection está saudável.

optimizer_status: ok

Não há problema detectado no processo de otimização.

points_count: 6

Os seis documentos foram inseridos.

segments_count: 8

Os dados estão distribuídos em segmentos internos. Em uma collection pequena e recém-escrita, isso não representa necessariamente problema.

indexed_vectors_count: 0

Isso não significa que os vetores estejam ausentes.

Na mesma resposta aparece:

```
1  "indexing_threshold": 10000
```

E

```
1  "full_scan_threshold": 10000
```

Como existem apenas seis pontos, o Qdrant não precisa construir HNSW para essa POC. Ele pode realizar busca exata ou varredura dos poucos pontos.

Para uma collection tão pequena:

- full scan é barato

- construir HNSW seria desnecessário

Portanto, o resultado é esperado:

- points_count = 6

- indexed_vectors_count = 0

- retrieval ainda funciona

14. O que a POC comprova sobre reingestão

A POC separa dois conceitos.

Ingestão documental

- abrir arquivo

- OCR

- extrair texto

- limpar

- dividir em chunks

- gerar metadados

- calcular checksum

Vetorização

- pegar texto já processado

- aplicar novo modelo

- salvar novo vetor

Se o payload contém:

```

1  {
2      "document_id": "...",
3      "chunk_id": "...",
4      "text": "...",
5      "checksum": "...",
6  }

```

é possível gerar um embedding novo diretamente desse texto.

Assim, evitamos:

- ✓ reabrir PDF
- ✓ executar OCR novamente
- ✓ limpar texto novamente
- ✓ recriar chunks
- ✓ reconstruir metadados

Mas ainda precisamos:

- ⚠ carregar o novo modelo
- ⚠ gerar embedding para cada chunk
- ⚠ armazenar os novos vetores

15. Limitação para evolução do schema

A POC cria os três named vectors desde o início.

Se posteriormente surgir um quarto modelo:

bge_m3_1024

a collection atual talvez não permita simplesmente adicionar um novo named vector inexistente por atualização. Registros recentes apontam que a inclusão de novos nomes vetoriais em uma collection existente pode exigir a criação de outra collection com o schema completo e migração dos pontos.

A estratégia segura é:

collection_v1

└— small

└─ base

└─ large

collection_v2

└─ small

└─ base

└─ large

└─ novo_modelo

Depois:

copiar payloads e vetores existentes

gerar apenas o embedding novo

validar benchmark

trocar alias

Isso evita reprocessar documentos, embora possa exigir migração entre collections.

LOGO,

Criamos uma collection do Qdrant com três named vectors. Cada vetor representa o mesmo documento por meio de um modelo de embedding diferente, com dimensões de 384, 768 e 1024. O payload textual é compartilhado, mas cada espaço vetorial tem dimensão e índice próprios. Durante a consulta, geramos a pergunta com cada modelo e indicamos ao Qdrant qual named vector deve ser pesquisado mediante o parâmetro using. Em seguida, comparamos os rankings individualmente e usamos Reciprocal Rank Fusion para obter um ranking consolidado sem misturar scores brutos de espaços incompatíveis. A POC comprova que conseguimos evitar novo OCR, parsing e chunking quando avaliamos outro modelo, embora ainda seja necessário calcular o embedding do novo modelo para cada chunk.

Questão	Conclusão
Uma collection aceita dimensões diferentes?	Sim, com named vectors
O mesmo ponto pode ter três embeddings?	Sim
Os scores são diretamente comparáveis?	Não
Como selecionar o modelo?	Com <code>using</code>
Como combinar os resultados?	RRF ou outra fusão de rankings
É necessário repetir parsing?	Não, se o texto estiver armazenado
É necessário repetir chunking?	Não, se os chunks estiverem versionados
É necessário calcular o novo embedding?	Sim
Um novo named vector pode exigir nova collection?	Sim
<code>indexed_vectors_count=0</code> significa ausência de vetores?	Não, nesse cenário pequeno